=== FILE: map_editor/tools/selection.py ===

```
"""Инструмент 'Выделение'"""

import pygame


class Selection:
    """Инструмент для выделения области"""

    def __init__(self, editor):
        self.editor = editor
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None
        self.is_selecting = False

    def start(self, x, y):
        """Начинает выделение"""
        self.start_x = x
        self.start_y = y
        self.end_x = x
        self.end_y = y
        self.is_selecting = True

    def update(self, x, y):
        """Обновляет выделение (при движении мыши)"""
        if self.is_selecting:
            self.end_x = x
            self.end_y = y

    def end(self):
        """Заканчивает выделение"""
        self.is_selecting = False

    def get_selection(self):
        """Возвращает выделенную область (x1, y1, x2, y2)"""
        if self.start_x is None or self.end_x is None:
            return None

        x1 = min(self.start_x, self.end_x)
        y1 = min(self.start_y, self.end_y)
        x2 = max(self.start_x, self.end_x)
        y2 = max(self.start_y, self.end_y)

        return (x1, y1, x2, y2)

    def get_cells(self):
        """Возвращает список клеток в выделенной области"""
        sel = self.get_selection()
        if not sel:
            return []

        x1, y1, x2, y2 = sel
        cells = []
        for y in range(y1, y2 + 1):
```

```
            for x in range(x1, x2 + 1):
                if 0 <= y < len(self.editor.grid) and 0 <= x < len(self.editor.grid[0]):
                    cells.append((x, y))
        return cells

    def fill(self, symbol):
        """Заполняет выделенную область символом"""
        cells = self.get_cells()
        if not cells:
            return

        changed = False
        for x, y in cells:
            if self.editor.grid[y][x] != symbol:
                self.editor.grid[y][x] = symbol
                changed = True

        if changed:
            self.editor._on_change()

        # Сбрасываем выделение
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None

    def clear(self):
        """Очищает выделенную область (ставит '_')"""
        self.fill('_')
        self.start_x = None
        self.start_y = None
        self.end_x = None
        self.end_y = None

    def draw(self, screen):
        """Рисует рамку выделения (вызывается из canvas)"""
        sel = self.get_selection()
        if not sel:
            return

        x1, y1, x2, y2 = sel
        # Преобразуем координаты клеток в пиксели
        rect_x = self.editor.canvas.rect.x + self.editor.canvas.scroll_x + x1 * self.editor.canvas.cell_si
ze
        rect_y = self.editor.canvas.rect.y + self.editor.canvas.scroll_y + y1 * self.editor.canvas.cell_si
ze
        rect_w = (x2 - x1 + 1) * self.editor.canvas.cell_size
        rect_h = (y2 - y1 + 1) * self.editor.canvas.cell_size

        # Рамка
        pygame.draw.rect(screen, (255, 255, 255), (rect_x, rect_y, rect_w, rect_h), 2)

        # Полупрозрачная заливка
        overlay = pygame.Surface((rect_w, rect_h))
        overlay.set_alpha(30)
        overlay.fill((255, 255, 255))
        screen.blit(overlay, (rect_x, rect_y))
```